

Edge cases (a missing required argument) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Story — `missing_argument_practice.py`: when a required argument is missing from the user's message, the model can ask, guess, or send an empty value — and which one it does tells you whether your tool needs to design for a clarifying question or a graceful default. **If not:** the Build Task's Pydantic argument models would be guessing at a problem you'd never actually watched happen.

Basic Version

Approach 1 — the direct way

```
# missing_argument_practice.py
from langchain_core.tools import tool
from langchain_openai import ChatOpenAI

@tool
def get_weather(city: str) -> str:
    """Get the current weather for a named city. Requires a city name."""
    return f"Sunny in {city}"

model_with_tools = ChatOpenAI().bind_tools([get_weather])

for run_number in range(1, 6):
    response = model_with_tools.invoke("what's the weather like")

    if not response.tool_calls:
        reply = response.content
        print(f"Run {run_number}: asked or answered directly -> {reply}")
        continue

    city_sent = response.tool_calls[0]["args"].get("city")
    if not city_sent:
        print(f"Run {run_number}: called with an empty city value")
```

```
else:  
    print(f"Run {run_number}: called with city = {city_sent!r}")
```

Expected output (an example split — real model behavior varies run to run):

```
Run 1: asked or answered directly -> Sure – which city do you mean?  
Run 2: called with city = 'London'  
Run 3: asked or answered directly -> Could you tell me which city you mean?  
Run 4: called with city = 'New York'  
Run 5: called with an empty city value
```

This shows you the real split across 5 runs. It doesn't save the results anywhere, so once the run finishes you've only got what scrolled past in the terminal — fine for a quick look, not for writing up what you found.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — classified and returned as structured records

```
# missing_argument_practice.py
from langchain_core.tools import tool
from langchain_openai import ChatOpenAI

@tool
def get_weather(city: str) -> str:
    """Get the current weather for a named city. Requires a city name."""
    return f"Sunny in {city}"

def get_model_with_tools() -> ChatOpenAI:
    model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
    return model.bind_tools([get_weather])

def classify_response(response) -> str:
    # why: sorts each run into one of 3 real outcomes instead of leaving it
    # to a human reading scrolled-past terminal text.
    if not response.tool_calls:
        return "asked_or_answered_directly"
    city_sent = response.tool_calls[0]["args"].get("city")
    if not city_sent:
        return "called_with_empty_value"
    return "called_with_guessed_value"

def run_missing_argument_trial(prompt: str, runs: int = 5) -> list[dict]:
    model_with_tools = get_model_with_tools()
    records = []

    for run_number in range(1, runs + 1):
        response = model_with_tools.invoke(prompt)
        outcome = classify_response(response)
        record = {"run": run_number, "outcome": outcome}
        if outcome == "called_with_guessed_value":
            record["city_sent"] = response.tool_calls[0]["args"].get("city")
```

```

        if outcome == "asked_or_answered_directly":
            record["reply"] = response.content
            records.append(record)

    return records

if __name__ == "__main__":
    results = run_missing_argument_trial("what's the weather like")
    for record in results:
        print(record)

```

Expected output (example):

```

{'run': 1, 'outcome': 'asked_or_answered_directly', 'reply': 'Which city?'}
{'run': 2, 'outcome': 'called_with_guessed_value', 'city_sent': 'London'}
{'run': 3, 'outcome': 'asked_or_answered_directly', 'reply': 'Specify a city?'}
{'run': 4, 'outcome': 'called_with_guessed_value', 'city_sent': 'New York'}
{'run': 5, 'outcome': 'called_with_empty_value'}

```

Difference from Basic: returning a list of typed records instead of only printing lets you actually count the split afterward (how many asked vs. guessed vs. something else) rather than having to re-read scrolled-past terminal output. Separating `classify_response()` out also makes it independently testable against a fake `response` object, no real API call needed.

Which one should you actually write? Intermediate's classified-records version is enough for this exercise — the point is watching and labeling what the model actually does with a missing argument, not fixing it. It's worth knowing this version never stops an empty-string guess from being accepted as a "real" value; a `Field(min_length=1)` constraint via `args_schema=` on the tool itself is the real fix, and costs almost nothing once you're ready to add it — but this exercise's job is observation, not the fix.